Project Title: Smart Automatic Water Tank Management and IoT Monitoring System

Developer / Name	Project Title	Target Microcontroller
Group 6 Students: V25BE9H1 V25BE9H2 V25BE9H3 V25BE9J1	**Smart Automatic Water Tank Management and IoT Monitoring System**	NXP LPC2129 (ARM7TDMI-S)

1. System Hardware Block Diagram (Pin-Mapped LPC2129 Architecture)

The hardware block diagram below maps all sensors, user switches, display/actuator outputs, and communication buses (UART0, I2C0, CAN1) according to the exact pin numbers configured in the LPC2129 firmware.

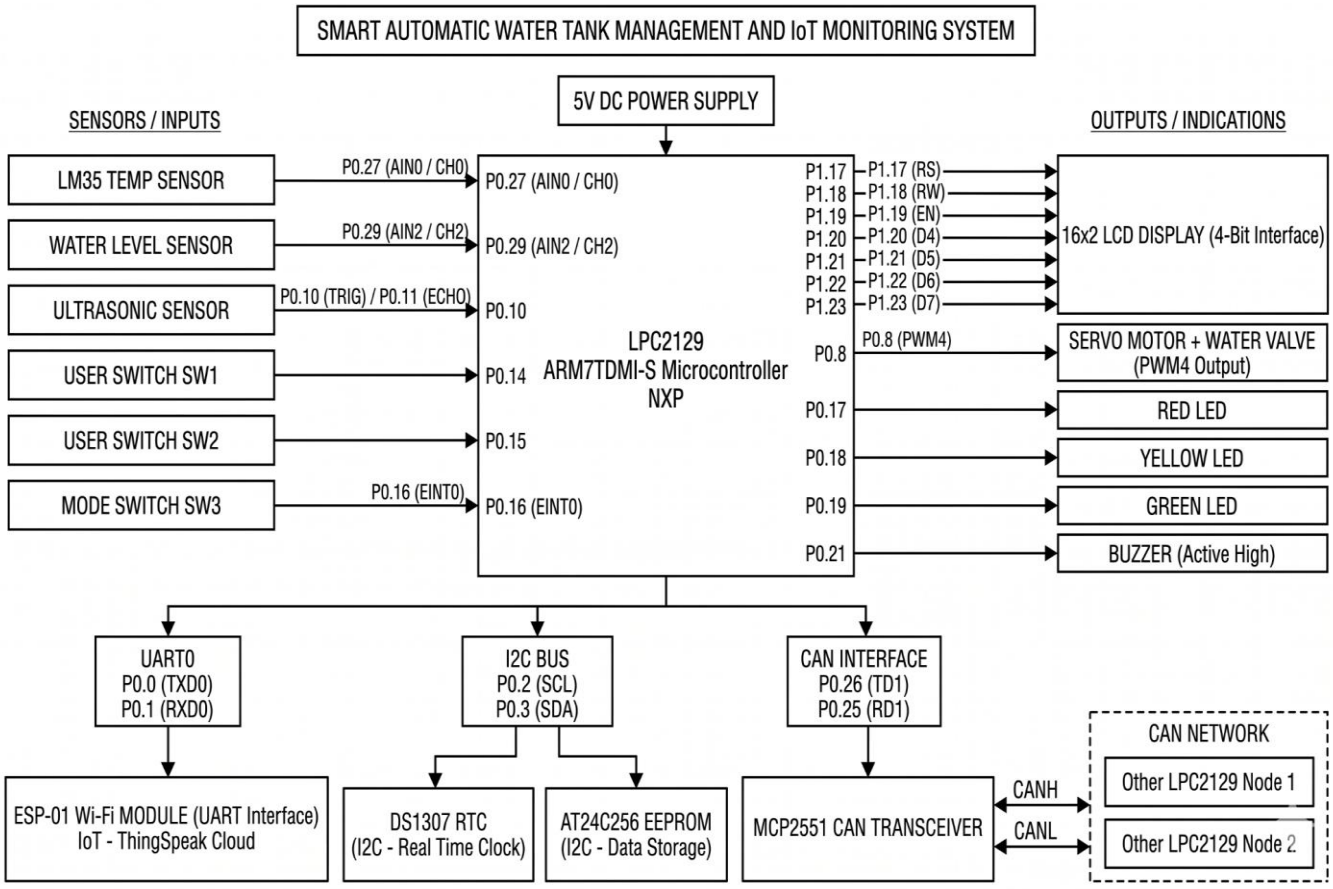


Figure 1.1: System Block Diagram of Smart Automatic Water Tank Management & IoT Monitoring System

2. Project Description & Working Principle

2.1 Abstract & System Overview

The Smart Automatic Water Tank Management and IoT Monitoring System is a real-time embedded automation and telemetry system built around the NXP LPC2129 (ARM7TDMI-S) 32-bit microcontroller. Conventional water storage systems suffer from frequent overflow wastage, dry-running motor failures, and lack of remote visibility. This project solves these issues by integrating multi-sensor level and temperature acquisition, hardware interrupt-driven Automatic/Manual operating modes, PWM servo valve control, DS1307 I2C Real-Time Clock (RTC) timestamping, AT24Cxx I2C EEPROM logging, CAN 2.0B multi-node broadcasting, and ESP-01 Wi-Fi cloud monitoring via ThingSpeak.

2.2 Detailed Working Principle of Hardware Subsystems

- **1. Analog & Ultrasonic Sensor Acquisition (ADC0 & Timer0):** The LPC2129 on-chip 10-bit successive-approximation ADC (ADCR = 0x00200400, PINSEL1 = 0x15400000) samples the Water Level Sensor on Channel 2 (P0.29 / AIN2) and converts the 10-bit digital value (0–1023) into a 0–100% tank level percentage. Channel 0 (P0.27 / AIN0) reads the analog voltage of the LM35 Temperature Sensor (10 mV/°C), converting it into degrees Celsius (vout * 100). Simultaneously, the HC-SR04 Ultrasonic Sensor uses P0.10 (TRIG) to send a 10 us trigger pulse and measures the high pulse duration on P0.11 (ECHO) using Timer0 microsecond delays (count / 58) to determine water surface distance in centimeters.
- **2. External Interrupt Mode Switching (EINT0 on P0.16):** Switch SW3 is wired to P0.16 (EINT0) and configured in the Vectored Interrupt Controller (VICVectCntl0 = 14 | (1<<5)) for falling-edge sensitivity. Pressing SW3 invokes EINT0_Handler(), immediately toggling the global mode variable between Automatic Mode (mode = 0) and Manual Mode (mode = 1).
- **3. Automatic Mode Operation (mode = 0):** When water_level < 20% (Low Level), the controller turns ON the Green LED (IOCLR0 = GREEN_LED), energizes the pump relay, and rotates the PWM4 Servo Motor (P0.8) to 90° (PWMMR4 = 1500, 1.5 ms pulse) to open the inlet water valve. When 20% <= water_level < 30%, the Yellow LED turns ON to indicate Medium Level. When water_level >= 30% (High Level), the active-high Buzzer (P0.21) sounds a 100 ms alarm, the Red LED turns ON, and the servo returns to 0° (PWMMR4 = 1000, 1.0 ms pulse) to close the valve and turn OFF the pump.
- **4. Manual Override Mode Operation (mode = 1):** In Manual Mode, the controller evaluates tank thresholds (< 20% Low, 20%–50% Medium, >= 50% High) and waits for physical operator confirmation. When the level is Low (< 20%), pressing active-low push button SW1 (P0.14) turns ON the pump and opens the servo valve (90°). When the level reaches High (>= 50%), the buzzer alerts the user and pressing SW2 (P0.15) closes the servo valve (0°) and turns OFF the pump.
- **5. I2C0 Real-Time Clock (DS1307) & Non-Volatile EEPROM (AT24Cxx):** The I2C0 Master peripheral (P0.2 SCL0, P0.3 SDA0) runs at 100 kbps (I2SCLH = I2SCLL = 75). At startup, main.c initializes the DS1307 RTC (0xD0) with time (12:58:55) and date (12/08/2026). During each monitoring loop, the MCU reads the BCD time and date registers (0xD1) and prints formatted timestamps over UART0. The eeprom.c driver logs temperature/status bytes to the AT24Cxx EEPROM (0xA0/0xA1) and performs read-back verification.
- **6. CAN 2.0B Network & ESP-01 Wi-Fi Cloud Telemetry:** The CAN1 controller (P0.25 RD1, P0.26 TD1) operates at 125 kbps (C1BTR = 0x001C001D with PCLK = 60 MHz) and packs temperature, water level, and ultrasonic distance into an 8-byte standard data frame (ID = 0x301 / 0x100) for remote CAN nodes. UART0 (P0.0 TXD0, P0.1 RXD0 at 9600 baud) transmits serial diagnostic reports and AT commands (AT+CWJAP, AT+CIPSTART, AT+CIPSEND) to an ESP-01 Wi-Fi module to push live telemetry to ThingSpeak Cloud.

2.3 Automatic vs. Manual Mode Threshold Matrix

Tank Level State	Automatic Mode (mode = 0)	Manual Mode (mode = 1)	Actuator & Indicator Action
LOW WATER LEVEL	Water Level < 20%	Water Level < 20% (Poll SW1 on P0.14)	Green LED ON | Pump ON | Servo Valve OPEN (90°)
MEDIUM WATER LEVEL	20% <= Water Level < 30%	20% <= Water Level < 50%	Yellow LED ON | Normal Filling / Monitoring

HIGH WATER LEVEL	Water Level >= 30%	Water Level >= 50% (Poll SW2 on P0.15)	Red LED ON | Buzzer Beep | Pump OFF | Valve CLOSE (0°)

3. Hardware Pin Allocation & Firmware Register Specifications

The following table summarizes all LPC2129 hardware pins, peripheral blocks, and register configurations used across the 12 source files of the project.

Hardware Module / Sensor	LPC2129 Pin	Peripheral Interface	Firmware Configuration & Role
LM35 Temperature Sensor	**P0.27 (AIN0 / CH0)**	10-Bit ADC0 (Ch 0)	PINSEL1 = 0x15400000; Analog temperature input (10mV/°C)
Water Level Sensor	**P0.29 (AIN2 / CH2)**	10-Bit ADC0 (Ch 2)	PINSEL1 = 0x15400000; Tank water level percentage (0–100%)
HC-SR04 Ultrasonic Sensor	**P0.10 (TRIG), P0.11 (ECHO)**	GPIO Port 0	10us Trigger pulse on P0.10; Echo width measured on P0.11
Manual Pump ON Switch (SW1)	**P0.14**	GPIO Input (Active-Low)	Polled in Manual Mode to turn ON Pump and open Valve (90°)
Manual Pump OFF Switch (SW2)	**P0.15**	GPIO Input (Active-Low)	Polled in Manual Mode to turn OFF Pump and close Valve (0°)
Mode Toggle Switch (SW3)	**P0.16 (EINT0)**	External Interrupt 0	Falling-edge ISR (VIC Ch 14) toggles Auto (0) & Manual (1) modes
Servo Motor (Water Valve)	**P0.8 (PWM4)**	Single-Edge PWM4	PINSEL0 (2<<16); 20ms period (PWMMR0=20000), 1.0ms–2.0ms pulse
Status Indicator LEDs	**P0.17 (R), P0.18 (Y), P0.19 (G)**	GPIO Output	Visual indication for Low (<20%), Medium, and High water levels
Piezo Buzzer Alarm	**P0.21**	GPIO Output (Active-High)	Triggers 100ms audible alert when high water threshold is reached
16x2 Alphanumeric LCD	**P1.17–P1.19 (Ctrl), P1.20–P1.23**	GPIO Port 1 (4-Bit Mode)	RS=P1.17, RW=P1.18, EN=P1.19; Data nibbles D4–D7 on P1.20–P1.23
ESP-01 Wi-Fi & Serial Terminal	**P0.0 (TXD0), P0.1 (RXD0)**	UART0 (9600 Baud, 8N1)	Streams telemetry logs & AT commands to ThingSpeak Cloud server
DS1307 RTC & AT24Cxx EEPROM	**P0.2 (SCL0), P0.3 (SDA0)**	I2C0 Master (100 kbps)	RTC (0xD0/0xD1) for time/date; EEPROM (0xA0/0xA1) for data logging
MCP2551 CAN Transceiver	**P0.25 (RD1), P0.26 (TD1)**	CAN1 Controller (125 kbps)	Transmits water level, temp & distance frames (ID 0x100 / 0x301)

3.1 Peripheral Timing & Register Calculations

- **Timer0 Precision Delay (delay.c):** With PCLK = 15 MHz, T0PR = 15000000 - 1 produces a 1-second tick (delay_sec), T0PR = 15000 - 1 produces a 1-millisecond tick (delay_ms), and T0PR = 14 produces a 1-microsecond tick (delay_us) for ultrasonic pulse measurement.

- **PWM4 Servo Motor Timing (servo.c):** PWMPR = 14 sets a 1 us prescaler tick. PWMMR0 = 20000 establishes a 20 ms period (50 Hz standard servo frame). PWMMR4 values of 1000 (1.0 ms), 1500 (1.5 ms), and 2000 (2.0 ms) position the valve at 0°, 90°, and 180° respectively.
- **I2C0 & CAN1 Bit-Rate Configuration (i2c.c, can_tx.c):** For 100 kbps I2C speed at 15 MHz PCLK, I2SCLH = I2SCLL = 75 (15 MHz / (75 + 75) = 100 kHz). For CAN1 at 60 MHz PCLK (VPBDIV = 1), C1BTR = 0x001C001D configures a 125 kbps industrial CAN bus bit rate.

4. Complete Modular Embedded C Source Code Listing

All 12 C and Header source code modules (.h and .c) are formatted below using individual line-by-line paragraphs so that every line breaks cleanly across pages in Microsoft Word, Google Docs, and PDF viewers.

1. header.h — *Macro Definitions, GPIO Pin Masks, CAN1 Structure & Function Prototypes*

```
#ifndef __HEADER_H__
#define __HEADER_H__

#include <LPC21xx.H>

#define RED_LED    (1<<17)
#define YELLOW_LED (1<<18)
#define GREEN_LED  (1<<19)
#define buz        (1<<21)

#define sw1        ((IOPIN0>>14)&1)
#define sw2        ((IOPIN0>>15)&1)

typedef unsigned int  u32;
typedef signed int    s32;
typedef unsigned char u8;
typedef signed char   s8;

typedef struct {
    u32 id;
    u32 rtr;
    u32 dlc;
    u32 byteA;
    u32 byteB;
} CAN1;

/* Timer0 Delay Prototypes */
void delay_sec(unsigned int sec);
void delay_ms(unsigned int m_s);
void delay_us(unsigned int ms);

/* UART0 Driver Prototypes */
void uart0_init(unsigned int baud);
void uart0_tx(unsigned char data);
unsigned char uart0_rx(void);
void uart0_tx_string(char *ptr);
void uart0_binary(int num);
void uart0_integer(int num);
void uart0_float(float f);

/* 16x2 LCD Driver Prototypes */
void lcd_init(void);
void lcd_cmd(unsigned int cmd);
void lcd_data(unsigned char data);
void lcd_string(char *ptr);
void lcd_integer(int num);
void lcd_float(float f);

/* ADC0 & Ultrasonic Sensor Prototypes */
void adc_init(void);
u32 adc_read(u8 ch_num);
void ultrasonic_init(void);
u32 ultrasonic_read(void);

/* Servo PWM4 & External Interrupt EINT0 Prototypes */
```

```
void servo_init(void);
void servo_0deg(void);
void servo_90deg(void);
void servo_180deg(void);
void EINT0_Handler(void) __irq;
void config_for_eint(void);

/* I2C0, EEPROM, CAN1 & ESP-01 Wi-Fi Prototypes */
void i2c_init(void);
void i2c_write(u8 sa, u8 mr, u8 data);
u8 i2c_read(u8 sa, u8 mr);
void eeprom(u8 t);
void can1_init(void);
void can1_tx(CAN1 v);
void can1_parameter(u8 t, u8 w, u8 d);
void wifi_connect(void);
void esp_read(void);

#endif
```

2. main.c — *Main Application Loop, Auto/Manual State Machine, RTC Read & Telemetry*

```
#include <LPC21xx.H>
#include "header.h"

extern int mode;
int t[6];

int main()
{
    int adc_value;
    u8 m, h, h1, s1, s, data;
    u8 d, mo, y;
    u32 distance = 62;
    float vout, adcout;
    float water_level, temp;
    unsigned int can_data[2];

    IODIR0 |= buz;
    IODIR0 |= RED_LED | YELLOW_LED | GREEN_LED;
    IOSET0  = RED_LED | YELLOW_LED | GREEN_LED;

    uart0_init(9600);
    adc_init();
    servo_init();
    lcd_init();
    i2c_init();
    ultrasonic_init();
    config_for_eint();
    //wifi_connect();
    //esp_read();

    uart0_tx_string("hello\r\n");

    /* Initialize DS1307 RTC Time & Date via I2C0 (Slave Addr: 0xD0) */
    i2c_write(0xD0, 0x02, 0x12); // Hours
    i2c_write(0xD0, 0x01, 0x58); // Minutes
    i2c_write(0xD0, 0x00, 0x55); // Seconds
    i2c_write(0xD0, 0x04, 0x12); // Date
    i2c_write(0xD0, 0x05, 0x08); // Month
    i2c_write(0xD0, 0x06, 0x26); // Year (2026)

    delay_ms(1000);
    //servo_180deg();
```

```
while(1)
{
    uart0_tx_string("\r\n=======================================\r\n");
    uart0_tx_string("      SMART WATER TANK MANAGEMENT       \r\n");
    uart0_tx_string("=======================================\r\n");
    uart0_tx_string("Controller: LPC2129 ARM7\r\n");
    uart0_tx_string("Project Status: RUNNING\r\n");

    distance = ultrasonic_read();

    adc_value = adc_read(2);
    water_level = (adc_value * 100) / 1023;
    uart0_tx_string("\r\n");
    uart0_integer(water_level);
    delay_sec(1);

    adcout = adc_read(0);
    vout = (adcout * 3.3) / 1023;
    temp = vout * 100;
    uart0_tx_string("\r\n");
    uart0_integer(adcout);
    delay_sec(1);

    if(mode == 1) // Odd: MANUAL MODE
    {
        uart0_tx_string("\r\nMANUAL MODE\r\n");
        if(water_level < 20)
        {
            uart0_tx_string("\r\nLOW WATER LEVEL\r\n");
            uart0_tx_string("Turn on the motor\r\n");
            IOSET0 = RED_LED | YELLOW_LED;
            IOCLR0 = GREEN_LED;
            while(1)
            {
                if(sw1 == 0)
                {
                    while(sw1 == 0);
                    uart0_tx_string("\r\nPUMP STATUS: ON\r\n");
                    //uart0_tx_string("\r\nRELAY STATUS: ON\r\n");
                    uart0_tx_string("\r\nVALVE STATUS: OPEN\r\n");
                    servo_90deg();
                    delay_ms(10);
                    uart0_tx_string("\r\nTemp=");
                    uart0_float(temp);
                    uart0_tx_string("%\r\n");
                    uart0_tx_string("Distance:");
                    uart0_float(distance);
                    uart0_tx_string("\r\n");
                    uart0_tx_string("Water level: ");
                    uart0_integer(water_level);
                    uart0_tx_string("\r\n");
                    delay_ms(50);
                    break;
                }
            }
        }
        else if(water_level >= 20 && water_level < 50)
        {
            uart0_tx_string("\r\nMEDIUM WATER LEVEL\r\n");
            IOSET0 = GREEN_LED | RED_LED;
            IOCLR0 = YELLOW_LED;
        }
        else if(water_level >= 50)
        {
```

```
            IOSET0 = buz;
            delay_ms(100);
            IOCLR0 = buz;
            uart0_tx_string("\r\nHIGH WATER LEVEL\r\n");
            uart0_tx_string("Turn off the motor\r\n");
            IOSET0 = GREEN_LED | YELLOW_LED;
            IOCLR0 = RED_LED;
            while(1)
            {
                if(sw2 == 0)
                {
                    while(sw2 == 0);
                    servo_0deg();
                    uart0_tx_string("\r\nPUMP STATUS: OFF\r\n");
                    uart0_tx_string("\r\nRELAY STATUS: OFF\r\n");
                    uart0_tx_string("\r\nVALVE STATUS: CLOSE\r\n");
                    delay_ms(100);
                    uart0_tx_string("\r\nTemp=");
                    uart0_float(temp);
                    uart0_tx_string("%\r\n");
                    uart0_tx_string("Water level: ");
                    uart0_integer(water_level);
                    uart0_tx_string("\r\n");
                    delay_ms(500);
                    uart0_tx_string("Distance:");
                    uart0_float(distance);
                    uart0_tx_string("\r\n");
                    break;
                }
            }
        }
    }
    else // Even: AUTOMATIC MODE
    {
        uart0_tx_string("\r\nAUTOMATIC MODE\r\n");
        if(water_level < 20)
        {
            uart0_tx_string("\r\nLOW WATER LEVEL\r\n");
            IOSET0 = RED_LED | YELLOW_LED;
            IOCLR0 = GREEN_LED;
            servo_90deg();
            uart0_tx_string("\r\nPUMP STATUS: ON\r\n");
            uart0_tx_string("\r\nRELAY STATUS: ON\r\n");
            uart0_tx_string("\r\nVALVE STATUS: OPEN\r\n");
            uart0_tx_string("\r\nTemp=");
            uart0_float(temp);
            uart0_tx_string("\r\n");
            uart0_tx_string("Distance:");
            uart0_float(distance);
            uart0_tx_string("\r\n");
            uart0_tx_string("Water level: ");
            uart0_integer(water_level);
            uart0_tx_string("\r\n");
            delay_ms(500);
        }
        else if(water_level >= 20 && water_level < 30)
        {
            uart0_tx_string("\r\nMEDIUM WATER LEVEL\r\n");
            IOSET0 = GREEN_LED | RED_LED;
            IOCLR0 = YELLOW_LED;
        }
        else if(water_level >= 30)
        {
            IOSET0 = buz;
            delay_ms(100);
            IOCLR0 = buz;
            uart0_tx_string("\r\nHIGH WATER LEVEL\r\n");
```

```
                uart0_tx_string("Turn off the motor\r\n");
                IOSET0 = GREEN_LED | YELLOW_LED;
                IOCLR0 = RED_LED;
                servo_0deg();
                uart0_tx_string("\r\nPUMP STATUS: OFF\r\n");
                uart0_tx_string("\r\nRELAY STATUS: OFF\r\n");
                uart0_tx_string("\r\nVALVE STATUS: CLOSE\r\n");
                uart0_tx_string("\r\nTemp=");
                uart0_float(temp);
                uart0_tx_string("%\r\n");
                uart0_tx_string("Water level: ");
                uart0_integer(water_level);
                uart0_tx_string("\r\n");
                delay_ms(500);
                uart0_tx_string("Distance:");
                uart0_float(distance);
                uart0_tx_string("\r\n");
            }
        }

        /* Read Date & Time from DS1307 RTC (0xD1) */
        uart0_tx_string("RTC Date: ");
        d  = i2c_read(0xD1, 0x04);
        mo = i2c_read(0xD1, 0x05);
        y  = i2c_read(0xD1, 0x06);
        uart0_tx(d/16 + 48);
        uart0_tx(d%16 + 48);
        uart0_tx('/');
        uart0_tx(mo/16 + 48);
        uart0_tx(mo%16 + 48);
        uart0_tx('/');
        uart0_tx_string("20");
        uart0_tx(y/16 + 48);
        uart0_tx(y%16 + 48);
        uart0_tx_string("\r\n");

        uart0_tx_string("RTC Time: ");
        h = i2c_read(0xD1, 0x02);
        m = i2c_read(0xD1, 0x01);
        s = i2c_read(0xD1, 0x00);
        h1 = h & 0x1F;
        uart0_tx(h1/16 + 48);
        uart0_tx(h1%16 + 48);
        uart0_tx(':');
        uart0_tx(m/16 + 48);
        uart0_tx(m%16 + 48);
        uart0_tx(':');
        uart0_tx(s/16 + 48);
        uart0_tx(s%16 + 48);
        if((h >> 5) & 1)
            uart0_tx_string(" PM\r\n");
        else
            uart0_tx_string(" AM\r\n");

        //eeprom(temp);
        //wifi_connect(); // P0.0 -> TX, P0.1 -> RX

        can_data[0] = (unsigned int)water_level;
        can_data[1] = (unsigned int)temp;
        can1_parameter((u8)temp, (u8)water_level, (u8)distance);
        delay_sec(5);
    }
}
```

3. adc.c — *10-Bit On-Chip ADC0 Initialization & Channel Conversion Driver (P0.27–P0.29)*

```
#include <LPC21xx.H>
#include "header.h"

void adc_init(void)
{
    PINSEL1 = 0x15400000; // Configure P0.27(AIN0), P0.28(AIN1), P0.29(AIN2)
    ADCR    = 0x00200400; // PDN=1 (Operational), CLKDIV=4
}

u32 adc_read(u8 ch_num)
{
    int temp;
    ADCR |= (1 << ch_num);          // Select ADC channel
    ADCR |= (1 << 24);              // Start A/D conversion now
    while(((ADDR >> 31) & 1) == 0); // Wait for DONE bit (Bit 31)
    ADCR ^= (1 << 24);              // Stop conversion
    ADCR ^= (1 << ch_num);          // Deselect channel
    temp = ((ADDR >> 6) & 0x3FF);   // Extract 10-bit result (Bits 15:6)
    return temp;
}
```

4. can_tx.c — *CAN1 Controller Initialization (125 kbps) & Frame Transmission Driver*

```
#include <LPC21xx.H>
#include "header.h"

void can1_init(void)
{
    VPBDIV  = 1;          // PCLK = 60 MHz
    PINSEL1 |= 0x00040000; // Configure P0.25 as RD1 (CAN1 RX)
    C1MOD   = 1;          // Select Reset Mode
    C1BTR   = 0x001C001D; // Configure 125 kbps baudrate
    AFMR    = 2;          // Accept all incoming messages (Bypass mode)
    C1MOD   = 0;          // Release Reset Mode (Operating Mode)
}

#define TCS ((C1GSR >> 3) & 1)

void can1_tx(CAN1 v)
{
    C1TID1 = v.id;
    C1TFI1 = (v.dlc << 16); // Set DLC, RTR=0, FF=0 (Standard 11-bit ID)
    if(v.rtr == 0)          // If Data Frame
    {
        C1TDA1 = v.byteA;
        C1TDB1 = v.byteB;
    }
    else
    {
        C1TFI1 |= (1 << 30); // Remote Transmission Request (RTR=1)
    }
    C1CMR = 0x21;            // Select TxBuf1 & Start Transmission
    while(TCS == 0);         // Wait until transmission is complete
}

void can1_parameter(u8 t, u8 w, u8 d)
{
    CAN1 MSG;
    MSG.id    = 0x301;
    MSG.rtr   = 0;
    MSG.dlc   = 8;
    MSG.byteA = 0;
    MSG.byteA |= t;
```

```
    MSG.byteA |= (w << 8);
    MSG.byteA |= (d << 16);
    MSG.byteB = 0;
    can1_tx(MSG);
}
```

5. delay.c — *Hardware Timer0 Precision Delay Routines (Seconds, Milliseconds, Microseconds)*

```
#include <LPC21xx.H>
#include "header.h"

void delay_sec(unsigned int sec)
{
    T0PR  = 15000000 - 1;
    T0TC  = 0;
    T0PC  = 0;
    T0TCR = 0x00;
    T0TCR = 0x01;
    while(T0TC < sec);
    T0TCR = 0x00;
}

void delay_ms(unsigned int m_s)
{
    T0PR  = 15000 - 1;
    T0TC  = 0;
    T0PC  = 0;
    T0TCR = 0x00;
    T0TCR = 0x01;
    while(T0TC < m_s);
    T0TCR = 0x00;
}

void delay_us(unsigned int ms)
{
    T0PR  = 14;
    T0TC  = 0;
    T0PC  = 0;
    T0TCR = 0x00;
    T0TCR = 0x01;
    while(T0TC < ms);
    T0TCR = 0x00;
}
```

6. eeprom.c — *AT24Cxx I2C EEPROM Data Logging & Read-Back Verification Module*

```
#include <LPC21xx.H>
#include "header.h"
s32 add;
void eeprom(u8 t)
{
    u8 temp;
    i2c_write(0xA0, add, t);
    delay_ms(30);
    temp = i2c_read(0xA1, add++);
    if(temp == t)
    {
        uart0_tx_string("\r\nEEPROM STATUS: DATA SAVED\r\n");
        uart0_tx(add/10 + 48);
        uart0_tx(add%10 + 48);
    }
    else
    {
        uart0_tx_string("\r\nEEPROM STATUS: DATA NOT SAVED\r\n");
        uart0_tx(add/10 + 48);
        uart0_tx(add%10 + 48);
```

```
    }
}
```

7. eint0.c — *External Interrupt 0 (EINT0 on P0.16) VIC Configuration & Mode Toggle ISR*

```
#include <LPC21xx.H>
#include "header.h"

int mode;

void EINT0_Handler(void) __irq
{
    mode = !mode;     // Toggle between Automatic (0) and Manual (1) mode
    EXTINT = 1;       // Clear EINT0 interrupt flag
    VICVectAddr = 0;  // Acknowledge end of ISR to VIC
}

void config_for_eint(void)
{
    PINSEL1     |= 1;                           // Configure P0.16 as EINT0
    EXTMODE     |= 1;                           // Edge-sensitive mode for EINT0
    EXTPOLAR    &= ~0;                          // Falling-edge sensitive
    VICIntSelect = 0;                           // Configure all interrupts as IRQ
    VICVectCntl0 = 14 | (1 << 5);               // Enable Slot 0 for EINT0 (Channel 14)
    VICVectAddr0 = (unsigned int)EINT0_Handler;  // Assign ISR address
    VICIntEnable = (1 << 14);                   // Enable EINT0 interrupt in VIC
}
```

8. i2c.c — *I2C0 Master Mode Driver (100 kbps) for DS1307 RTC & AT24Cxx EEPROM*

```
#include <LPC21xx.H>
#include "header.h"

void i2c_init(void)
{
    PINSEL0  |= 0x50;     // P0.2 -> SCL0, P0.3 -> SDA0
    I2CONSET  = (1 << 6); // Enable I2C interface (I2EN=1, Master Mode)
    I2SCLH    = I2SCLL = 75; // 100 kbps standard clock frequency
}

#define SI ((I2CONSET >> 3) & 1)

void i2c_write(u8 sa, u8 mr, u8 data)
{
    /* 1. Generate START condition */
    I2CONSET = (1 << 5); // STA = 1
    I2CONCLR = (1 << 3); // Clear SI
    while(SI == 0);      // Wait for START condition
    I2CONCLR = (1 << 5); // STA = 0

    /* 2. Send Slave Address + Write (SA+W) & check ACK */
    I2DAT    = sa;       // Send SA+W
    I2CONCLR = (1 << 3); // Clear SI
    while(SI == 0);
    if(I2STAT == 0x20)
    {
        uart0_tx_string("\r\nErr: SA+W\r\n");
        goto exit;
    }

    /* 3. Send Memory/Register Address & check ACK */
    I2DAT    = mr;       // Send memory location
    I2CONCLR = (1 << 3); // Clear SI
    while(SI == 0);
```

```
    if(I2STAT == 0x30)
    {
        uart0_tx_string("\r\nErr: Memory Addr\r\n");
        goto exit;
    }

    /* 4. Send Data Byte & check ACK */
    I2DAT    = data;     // Send data
    I2CONCLR = (1 << 3); // Clear SI
    while(SI == 0);
    if(I2STAT == 0x30)
    {
        uart0_tx_string("\r\nErr: Data\r\n");
        goto exit;
    }

    /* 5. Generate STOP condition */
exit:
    I2CONSET = (1 << 4); // STO = 1
    I2CONCLR = (1 << 3); // Clear SI
}

u8 i2c_read(u8 sa, u8 mr)
{
    u8 temp;

    /* 1. Generate START condition */
    I2CONSET = (1 << 5); // STA = 1
    I2CONCLR = (1 << 3); // Clear SI
    while(SI == 0);
    I2CONCLR = (1 << 5); // STA = 0

    /* 2. Send SA+W & check ACK */
    I2DAT    = sa & 0xFE; // Send SA+W
    I2CONCLR = (1 << 3);  // Clear SI
    while(SI == 0);
    if(I2STAT == 0x20)
    {
        uart0_tx_string("\r\nErr: SA+W\r\n");
        goto exit;
    }

    /* 3. Send Memory Address & check ACK */
    I2DAT    = mr;
    I2CONCLR = (1 << 3);
    while(SI == 0);
    if(I2STAT == 0x30)
    {
        uart0_tx_string("\r\nErr: Memory Addr\r\n");
        goto exit;
    }

    /* 4. Generate Repeated START condition */
    I2CONSET = (1 << 5); // STA = 1
    I2CONCLR = (1 << 3); // Clear SI
    while(SI == 0);
    I2CONCLR = (1 << 5); // STA = 0

    /* 5. Send SA+R & check ACK */
    I2DAT    = sa;       // Send SA+R
    I2CONCLR = (1 << 3);
    while(SI == 0);
    if(I2STAT == 0x48)
    {
```

```
        uart0_tx_string("\r\nErr: SA+R\r\n");
        goto exit;
    }

    /* 6. Read Data Byte & send NACK */
    I2CONCLR = (1 << 3);
    while(SI == 0);
    temp = I2DAT;

    /* 7. Generate STOP condition */
exit:
    I2CONSET = (1 << 4); // STO = 1
    I2CONCLR = (1 << 3); // Clear SI

    /* 8. Return received byte */
    return temp;
}
```

9. lcd.c — *16x2 Alphanumeric LCD 4-Bit Driver (Port 1: P1.17–P1.23)*

```
#include <LPC21xx.H>
#include "header.h"

void lcd_data(unsigned char data)
{
    unsigned int temp;
    IOCLR1 = 0xFE << 16;
    temp   = ((data & 0xF0) << 16);
    IOSET1 = temp;
    IOSET1 = (1 << 17); // RS = 1 (Data Register)
    IOCLR1 = (1 << 18); // RW = 0 (Write)
    IOSET1 = (1 << 19); // EN = 1
    delay_ms(2);
    IOCLR1 = (1 << 19); // EN = 0

    IOCLR1 = 0xFE << 16;
    temp   = ((data & 0x0F) << 20);
    IOSET1 = temp;
    IOSET1 = (1 << 17); // RS = 1
    IOCLR1 = (1 << 18); // RW = 0
    IOSET1 = (1 << 19); // EN = 1
    delay_ms(2);
    IOCLR1 = (1 << 19); // EN = 0
}

void lcd_cmd(unsigned int cmd)
{
    unsigned int temp;
    IOCLR1 = 0xFE << 16;
    temp   = ((cmd & 0xF0) << 16);
    IOSET1 = temp;
    IOCLR1 = (1 << 17); // RS = 0 (Command Register)
    IOCLR1 = (1 << 18); // RW = 0
    IOSET1 = (1 << 19); // EN = 1
    delay_ms(2);
    IOCLR1 = (1 << 19); // EN = 0

    IOCLR1 = 0xFE << 16;
    temp   = ((cmd & 0x0F) << 20);
    IOSET1 = temp;
    IOCLR1 = (1 << 17); // RS = 0
    IOCLR1 = (1 << 18); // RW = 0
    IOSET1 = (1 << 19); // EN = 1
    delay_ms(2);
    IOCLR1 = (1 << 19); // EN = 0
```

```
}

void lcd_init(void)
{
    IODIR1 = 0xFE << 16; // Configure P1.17 to P1.23 as output
    lcd_cmd(0x02);       // Return Home (4-bit initialization)
    lcd_cmd(0x28);       // 4-bit mode, 2 lines, 5x7 matrix
    lcd_cmd(0x0E);       // Display ON, Cursor ON
    lcd_cmd(0x01);       // Clear Display
}

void lcd_string(char *ptr)
{
    while(*ptr != '\0')
    {
        lcd_data(*ptr);
        ptr++;
    }
}

void lcd_integer(int num)
{
    int a[10], i;
    if(num == 0)
        lcd_data('0');
    if(num < 0)
    {
        num = -num;
        lcd_data('-');
    }
    for(i = 0; num > 0; num /= 10, i++)
        a[i] = (num % 10) + 48;
    for(i = i - 1; i >= 0; i--)
        lcd_data(a[i]);
}

void lcd_float(float f)
{
    int num, n1;
    if(f == 0)
    {
        lcd_data('0');
        return;
    }
    if(f < 0)
    {
        f = -f;
        lcd_data('-');
    }
    num = f;
    lcd_integer(num);
    lcd_data('.');
    n1 = (f - num) * 100000;
    lcd_integer(n1);
}
```

10. servo.c — *PWM4 Hardware Servo Valve Driver on P0.8 (0°, 90°, 180° Control)*

```
#include <LPC21xx.H>
#include "header.h"

#define SERVO (1 << 8) // P0.8 -> PWM4

void servo_init(void)
{
```

```
    PINSEL0 = (PINSEL0 & ~(3 << 16)) | (2 << 16); // Configure P0.8 as PWM4
    PWMTCR  = (1 << 1);          // Counter Reset
    PWMPR   = 14;                // Prescaler for 1 us tick at 15 MHz PCLK
    PWMMR0  = 20000;             // 20 ms PWM period (50 Hz)
    PWMMR4  = 1500;              // Default 1.5 ms pulse (90 deg)
    PWMMCR  = (1 << 1);          // Reset PWMTC on PWMMR0 match
    PWMLER  = (1 << 0) | (1 << 4); // Enable shadow latch for MR0 & MR4
    PWMPCR  = (1 << 12);         // Enable PWM4 single-edge output
    PWMTCR  = (1 << 0) | (1 << 3); // Enable Counter and PWM Mode
}

void servo_0deg(void)
{
    PWMMR4 = 1000;       // 1.0 ms pulse width -> 0 degrees (Valve Closed)
    PWMLER = (1 << 4);
}

void servo_90deg(void)
{
    PWMMR4 = 1500;       // 1.5 ms pulse width -> 90 degrees (Valve Open)
    PWMLER = (1 << 4);
}

void servo_180deg(void)
{
    PWMMR4 = 2000;       // 2.0 ms pulse width -> 180 degrees
    PWMLER = (1 << 4);
}
```

11. uart.c — *UART0 Serial Communication Driver (P0.0 TXD0, P0.1 RXD0)*

```
#include <LPC21xx.H>
#include "header.h"

void uart0_init(unsigned int baud)
{
    int pclk, result = 0;
    if(VPBDIV == 0)
        pclk = 15000000;
    else if(VPBDIV == 1)
        pclk = 60000000;
    else if(VPBDIV == 2)
        pclk = 30000000;

    result   = pclk / (16 * baud);
    PINSEL0 |= 0x00000005;       // Configure P0.0 as TXD0, P0.1 as RXD0
    U0LCR    = 0x83;             // 8-bit data, 1 stop bit, DLAB = 1
    U0DLL    = (result & 0xFF);
    U0DLM    = ((result >> 8) & 0xFF);
    U0LCR    = 0x03;             // Clear DLAB = 0
}

#define THRE ((U0LSR >> 5) & 1)

void uart0_tx(unsigned char data)
{
    U0THR = data;
    while(THRE == 0);
}

#define RDR (U0LSR & 1)

unsigned char uart0_rx(void)
{
```

```
    while(RDR == 0);
    return U0RBR;
}

void uart0_tx_string(char *ptr)
{
    while(*ptr != '\0')
    {
        uart0_tx(*ptr);
        ptr++;
    }
}

void uart0_binary(int num)
{
    int pos, temp;
    for(pos = 31; pos >= 0; pos--)
    {
        temp = ((num >> pos) & 1);
        uart0_tx(temp + 48);
    }
}

void uart0_integer(int num)
{
    int a[20], i;
    if(num < 0)
    {
        num = -num;
        uart0_tx('-');
    }
    if(num == 0)
    {
        uart0_tx('0');
        return;
    }
    for(i = 0; num > 0; num /= 10, i++)
        a[i] = (num % 10) + 48;
    for(i = i - 1; i >= 0; i--)
        uart0_tx(a[i]);
}

void uart0_float(float f)
{
    int num, n1;
    if(f == 0)
    {
        uart0_tx('0');
        return;
    }
    if(f < 0)
    {
        f = -f;
        uart0_tx('-');
    }
    num = (int)f;
    uart0_integer(num);
    uart0_tx('.');
    n1 = (f - num) * 100;
    uart0_integer(n1);
}
```

12. ultrasonic.c — *HC-SR04 Ultrasonic Distance Sensor Driver (P0.10 TRIG, P0.11 ECHO)*

```
#include <LPC21xx.H>
#include "header.h"
```

```
#define TRIG (1 << 10) // P0.10 -> Trigger Output
#define ECHO (1 << 11) // P0.11 -> Echo Input

void ultrasonic_init(void)
{
    IODIR0 |= TRIG;
    IODIR0 &= ~ECHO;
}

u32 ultrasonic_read(void)
{
    u32 count = 0;
    IOCLR0 = TRIG;
    delay_us(2);
    IOSET0 = TRIG;
    delay_us(10);
    IOCLR0 = TRIG;

    while(!(IOPIN0 & ECHO)); // Wait for Echo HIGH
    while(IOPIN0 & ECHO)     // Measure Echo HIGH pulse width in us
    {
        count++;
        delay_us(1);
        if(count > 30000)
        {
            uart0_tx_string("timeout\r\n");
            return 0;
        }
    }
    return (count / 58);     // Convert pulse duration to distance in cm
}
```

13. wifi.c — *ESP-01 Wi-Fi Module AT Command & ThingSpeak Cloud Telemetry Driver*

```
#include <LPC21xx.H>
#include "header.h"

void wifi_connect(void) // P0.0 -> TXD0, P0.1 -> RXD0
{
    uart0_tx_string("AT\r\n");
    delay_ms(2000);
    uart0_tx_string("AT+CWMODE=1\r\n");
    delay_ms(2000);
    uart0_tx_string("AT+CWJAP=\"Samsung\",\"91919191\"\r\n");
    delay_ms(20000);
    uart0_tx_string("AT+CIFSR\r\n");
    delay_ms(2000);
    uart0_tx_string("AT+CIPSTART=\"TCP\",\"api.thingspeak.com\",80\r\n");
    delay_ms(5000);
    uart0_tx_string("AT+CIPSEND=122\r\n");
    delay_ms(20000);
    uart0_tx_string("GET            /update?api_key=K0KJB6VB538JT2WJ&field1=50&field2=120&field3=28&field4=1
HTTP/1.1\r\n");
    delay_ms(5000);
    uart0_tx_string("Host:api.thingspeak.com\r\n\r\n");
    delay_ms(5000);
    uart0_tx_string("Connection:close\r\n\r\n");
    delay_ms(8000);
}

void esp_read(void)
{
    unsigned char ch;
    while(1)
```

```
    {
        ch = uart0_rx();
        uart0_tx(ch);
    }
}
```